

# Guía de Estudio Definitiva: Machine Learning (Semanas 9-15)

He auditado tus notebooks de las **Semanas 9 a la 15** e inyectado comentarios exhaustivos de "Lógica de Examen" en el código fuente de los notebooks centrales de cada tema.

Además, he analizado el nuevo notebook que agregaste en la Semana 12, [16\\_01 ML Segmentacion de Clientes.ipynb](#). Es el notebook de Clustering K-Means más completo que tienes, por lo que lo seleccioné como el notebook principal para ese tema y le inyecté explicaciones detalladas sobre el escalado, el método del codo y el análisis de siluetas.

A continuación, tienes la **guía teórica definitiva** de la segunda mitad del curso, diseñada para que no te saltes ninguna explicación y comprendas cada algoritmo de cara a tu examen.

---

## 1. Semana 9: Support Vector Machines (SVM) y Balanceo (SMOTE)

### Support Vector Machines (SVM)

- **¿Cómo funciona?** Busca encontrar un **hiperplano óptimo** que separe dos clases en un espacio multidimensional. El objetivo es que este hiperplano tenga el **máximo margen** posible (distancia más grande entre el hiperplano y los puntos más cercanos de cada clase, llamados **vectores de soporte**).
- **Margen Duro (Hard Margin) vs. Margen Blando (Soft Margin):**
  - *Hard Margin*: Asume que los datos son perfectamente separables. No tolera ningún error de clasificación.
  - *Soft Margin*: Permite que algunos puntos violen el margen para lograr un modelo que generalice mejor. Esto se controla con el parámetro **C**.
- **Hiperparámetros Clave:**
  - **C** (Regularización): Controla el balance entre maximizar el margen y minimizar el error de clasificación.
    - **C alto**: Penaliza fuertemente las clasificaciones erróneas. Intenta clasificar todo bien, estrechando el margen. Riesgo de **Overfitting** (sobreajuste).
    - **C bajo**: Permite errores de clasificación a cambio de un margen más ancho. Generaliza mejor. Riesgo de **Underfitting** (subajuste).
  - **kernel** (El Truco del Kernel): Si los datos no son linealmente separables en el espacio original, el kernel los proyecta matemáticamente a una dimensión superior donde sí sean linealmente separables.
    - **linear**: Para datos que se pueden separar con una línea recta.
    - **rbf** (Radial Basis Function): Crea fronteras de decisión esféricas y complejas. Es el más usado por defecto.
    - **poly** (Polinomial): Crea fronteras curvadas de grado  $d$ .

- **gamma** (Exclusivo de kernels no lineales como RBF): Define el radio de influencia de un solo ejemplo de entrenamiento.
  - **Gamma alto:** Un punto influye solo en sus vecinos más cercanos. La frontera se vuelve muy irregular y se ajusta a cada punto individual (**Overfitting**).
  - **Gamma bajo:** Un punto tiene una influencia de gran alcance. La frontera de decisión es más suave y lisa (**Underfitting**).

[!IMPORTANT] **¿Por qué exige StandardScaler?** SVM calcula distancias geométricas euclidianas desde los puntos al hiperplano. Si una variable tiene un rango de \$0\$ a \$1,000,000\$ (ej: Salario) y otra de \$0\$ a \$10\$ (ej: Años de estudio), el algoritmo ignorará por completo los años de estudio. Escalarlos a media 0 y varianza 1 es mandatorio.

## Balanceo con SMOTE

- **¿Por qué lo usamos?** Cuando hay un desbalance severo (ej: 99% transacciones normales, 1% fraude), el clasificador simplemente predecirá "normal" siempre para tener un Accuracy ficticio de 99%.
- **SMOTE (Synthetic Minority Over-sampling Technique):** Crea ejemplos **sintéticos** de la clase minoritaria en lugar de solo duplicar observaciones existentes (lo que causaría sobreajuste).
- **¿Cómo funciona?** Toma una muestra de la clase minoritaria, busca sus  $k$  vecinos más cercanos de la misma clase, dibuja una línea entre ellos y crea un punto sintético intermedio en esa línea.
- **Regla de Oro:** El SMOTE **solo se aplica a los datos de entrenamiento (Train)**. Nunca a los de prueba (Test) ni validación, ya que queremos medir el desempeño real bajo la distribución desbalanceada original.

## 2. Semana 10: Ensamblados (Voting, Bagging, Boosting, Random Forest)

Un ensamble combina múltiples modelos para obtener una predicción más robusta y precisa que cualquier modelo individual.

### Voting Classifier (Votación)

Combina algoritmos heterogéneos (ej: une un SVM, un KNN y un Random Forest).

- **Hard Voting (Voto Mayoritario):** Cada clasificador predice una clase (0 o 1). La clase que obtenga más votos del jurado es la predicción final.
- **Soft Voting (Voto Ponderado):** Se promedian las probabilidades de pertenecer a cada clase dadas por cada clasificador. La clase con el promedio de probabilidad más alto gana. **Requiere que todos los clasificadores puedan dar probabilidades (predict\_proba).** Es generalmente superior porque valora la certeza del clasificador.

## Bagging (Bootstrap Aggregating)

- **¿Cómo funciona?** Entrena múltiples instancias del **mismo algoritmo base** (típicamente árboles de decisión sin podar) en paralelo. Cada modelo se entrena con un subconjunto de datos obtenido mediante **Bootstrap** (muestreo aleatorio con reemplazo).
- **Objetivo:** Su meta es **reducir la varianza** (evitar el Overfitting) promediando las predicciones (o votando en clasificación).
- **Random Forest:** Es un caso especial de Bagging donde, además de muestrear filas de datos, en cada división de nodo del árbol se selecciona un **subconjunto aleatorio de variables (columnas)**. Esto descorrelaciona los árboles, logrando que el ensamble sea aún más diverso y robusto.

## Boosting

- **¿Cómo funciona?** Entrena modelos de forma **secuencial** (uno después del otro). Cada nuevo modelo se enfoca en corregir los errores cometidos por los modelos anteriores, asignándoles pesos mayores a los datos que fueron mal clasificados.
- **AdaBoost (Adaptive Boosting):** Usa clasificadores muy débiles (árboles de profundidad 1 o "decision stumps"). En cada iteración, incrementa la importancia de las observaciones difíciles de clasificar.
- **Objetivo:** Su meta principal es **reducir el sesgo** (ayuda a que el modelo aprenda patrones complejos que un árbol simple no vería).

[!NOTE] **Métrica en Random Forest: feature\_importances\_** Mide cuánto contribuye cada variable a reducir la impureza (Gini o Entropía) en los nodos de los árboles. Suma 1.0 en total y sirve para hacer selección de variables (Feature Selection).

---

## 3. Semana 11: Regresiones (Lineal y Logística)

### Regresión Lineal

- **¿Qué hace?** Predice un valor numérico continuo.
- **Ecuación:**  $y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots$ 
  - $\beta_0$  (Intercepto): Valor de  $y$  cuando todas las  $x$  son 0.
  - $\beta_1$  (Coeficiente/Pendiente): El cambio estimado en  $y$  por cada unidad que aumenta  $x_1$ .
- **Evaluación:**
  - **MSE (Mean Squared Error):** Promedio de los errores al cuadrado. Penaliza fuertemente las desviaciones grandes.
  - **$R^2$  (Coeficiente de Determinación):** Representa la proporción de la varianza de la variable objetivo que es explicada por el modelo (de 0 a 1). Un  $R^2 = 0.85$  significa que el modelo explica el 85% del comportamiento de los datos.

## Regresión Logística

- **OJO:** Aunque lleva "Regresión" en su nombre, **es un algoritmo de Clasificación Binaria**.
  - **¿Cómo funciona?** Toma la salida de una ecuación lineal y la pasa por la **Función Sigmoide** para transformarla en una probabilidad entre 0 y 1:  $S(z) = \frac{1}{1 + e^{-z}}$
  - **Umbral:** Por defecto, si  $S(z) \geq 0.5$ , el modelo predice la clase 1. Si no, predice la clase 0.
- 

## 4. Semana 12: Aprendizaje No Supervisado: Clustering (K-Means y Jerárquico)

El objetivo es agrupar datos similares en "clusters" sin tener etiquetas preestablecidas.

### K-Means Clustering

- **¿Cómo funciona?**
  1. Elige aleatoriamente  $K$  puntos como centroides iniciales.
  2. Asigna cada dato al centroide más cercano (usando distancia euclidiana).
  3. Actualiza la posición del centroide calculando la media de todos los puntos asignados a él.
  4. Repite hasta que los centroides no cambien de posición.
- **Sensibilidad:** Exige `StandardScaler` porque depende de distancias geométricas.

### ¿Cómo determinar el "K" óptimo?

Los profesores de la UPC aman esta pregunta. Tienes dos herramientas:

1. **Método del Codo (Elbow Method) - Basado en la Inercia (WCSS):**
  - *Inercia / WCSS:* La suma de las distancias al cuadrado de cada punto a su centroide asignado.
  - *Criterio:* A medida que aumentas  $K$ , la inercia disminuye. Graficas  $K$  vs. Inercia y buscas el **punto de inflexión o codo** donde la inercia deja de bajar abruptamente.
2. **Coeficiente de Silueta (Silhouette Score):**
  - *Qué es:* Mide qué tan similar es un punto a su propio cluster (cohesión) comparado con el cluster vecino más cercano (separación).
  - *Rango:*  $-1$  a  $1$ .
    - Cercano a 1: El punto está muy bien agrupado.
    - Cercano a 0: El punto está en el límite entre dos clusters.
    - Negativo: El punto probablemente pertenece al otro cluster.
  - *Criterio:* Elegimos el  $K$  que maximice el coeficiente de silueta promedio.

## Clustering Jerárquico (Agglomerative)

- **¿Cómo funciona?** Comienza tratando a cada dato como un cluster individual y los va fusionando secuencialmente según su cercanía hasta formar un único gran cluster.
  - **Dendrograma:** Gráfico en forma de árbol que ilustra las fusiones. Cortar el dendrograma horizontalmente define el número final de clusters.
  - **Criterio de Enlace (Linkage):**
    - **ward:** Minimiza la varianza dentro de los clusters fusionados (el más común).
    - **complete:** Distancia máxima entre puntos de dos clusters.
    - **single:** Distancia mínima entre puntos de dos clusters.
- 

## 5. Semana 13: Reglas de Asociación (Algoritmo Apriori)

Busca patrones frecuentes en transacciones (ej: compra de supermercado: *quien compra hamburguesas suele comprar papas fritas*).

### Métricas Clave

Sea una regla  $A \rightarrow B$  ( $A$  es el antecedente,  $B$  es el consecuente):

1. **Soporte (Support):** Frecuencia relativa con la que el conjunto de ítems aparece en todas las transacciones.  $[\text{Soporte}(A \cup B) = \frac{\text{Transacciones con A y B}}{\text{Total de transacciones}}]$
  2. **Confianza (Confidence):** Medida de certeza. De las transacciones que contienen A, ¿cuántas contienen también B?  $[\text{Confianza}(A \rightarrow B) = \frac{\text{Soporte}(A \cup B)}{\text{Soporte}(A)}]$
  3. **Sustento (Lift):** Mide la fuerza de la regla comparada con el azar (si A y B fueran independientes).  $[\text{Lift}(A \rightarrow B) = \frac{\text{Soporte}(A \cup B)}{\text{Soporte}(A) \times \text{Soporte}(B)}]$ 
    - **Lift > 1:** Asociación positiva. Comprar A incrementa la probabilidad de comprar B.
    - **Lift = 1:** Independencia. A y B se compran juntos por pura casualidad.
    - **Lift < 1:** Asociación negativa (sustitución). Comprar A disminuye la probabilidad de comprar B.
- 

## 6. Semana 14: Aprendizaje Semi-Supervisado

- **¿Cuándo se usa?** Cuando etiquetar datos es muy costoso y tenemos un mar de datos no etiquetados y muy pocos datos etiquetados.
- **Enfoque con Clustering (Active Learning + Label Propagation):**
  1. Corremos K-Means sobre todo el dataset (ej: 50 clusters).
  2. Identificamos los puntos más cercanos al centroide de cada cluster. Estos son los

**dígitos representativos** (representan la esencia del grupo).

3. Un experto humano etiqueta a mano únicamente esos 50 ejemplos (ahorramos costo de etiquetado).
  4. **Propagación de etiquetas (Label Propagation):** Asignamos esa misma etiqueta humana a todos los demás puntos que cayeron en ese mismo cluster.
  5. Entrenamos un clasificador supervisado convencional (ej: Regresión Logística) sobre todo el dataset ahora pseudo-etiquetado.
- **Self-Training:** El modelo se entrena con los pocos datos etiquetados. Luego predice sobre los no etiquetados y añade a su conjunto de entrenamiento aquellos donde su predicción tenga una certeza muy alta (ej:  $>95\%$ ).
- 

## 7. Semana 15: Aprendizaje por Refuerzo (Reinforcement Learning)

- **Agente y Entorno:** El **Agente** (quien toma decisiones) interactúa con el **Entorno** (el mundo en el que opera) ejecutando **Acciones** que cambian el **Estado** y recibe a cambio una **Recompensa** (positiva o negativa). Su meta es maximizar la recompensa acumulada.
- **Q-Learning:** Algoritmo que mantiene una **Q-Table**, donde cada celda guarda el valor Q de ejecutar la acción  $a$  en el estado  $s$ .
- **Ecuación de Bellman (Actualización del valor Q):** 
$$Q(s, a) \leftarrow Q(s, a) + \alpha [R + \gamma \max_{a'} Q(s', a') - Q(s, a)]$$
  - $\alpha$  (Tasa de Aprendizaje): Controla qué tanto peso tiene la nueva experiencia frente al conocimiento histórico.
  - $\gamma$  (Factor de Descuento): Controla el valor de las recompensas futuras.  $\gamma = 0$  hace al agente miope (solo le importa el premio inmediato),  $\gamma = 1$  lo hace visionario (le importan las recompensas a largo plazo).
  - $\epsilon$  (Exploración vs. Explotación):
    - *Exploración:* Tomar acciones aleatorias para descubrir nuevos caminos y recompensas.
    - *Explotación:* Tomar la acción que el modelo ya sabe que da la recompensa más alta (según la Q-Table).
    - *Epsilon-Greedy:* Con probabilidad  $\epsilon$  el agente explora; con probabilidad  $1 - \epsilon$  explota. Epsilon se va reduciendo a medida que el agente aprende.